

CvSU Vision

The premier university in historic Cavite globally recognized for excellence in character development, academics, research, innovation and sustainable community engagement.



CAVITE STATE UNIVERSITY

Bacoor City Campus
Soldiers Hills IV, Molino VI,
City of Bacoor, Cavite
(046) 476 - 5029
www.cvsu.edu.ph

CvSU Mission

Cavite State University shall provide excellent, equitable and relevant educational opportunities in the arts, science and technology through quality instruction and responsive research and development activities.
It shall produce professional, skilled and morally upright individuals for global competitiveness.

DEPARTMENT OF COMPUTER STUDIES
DCIT 26

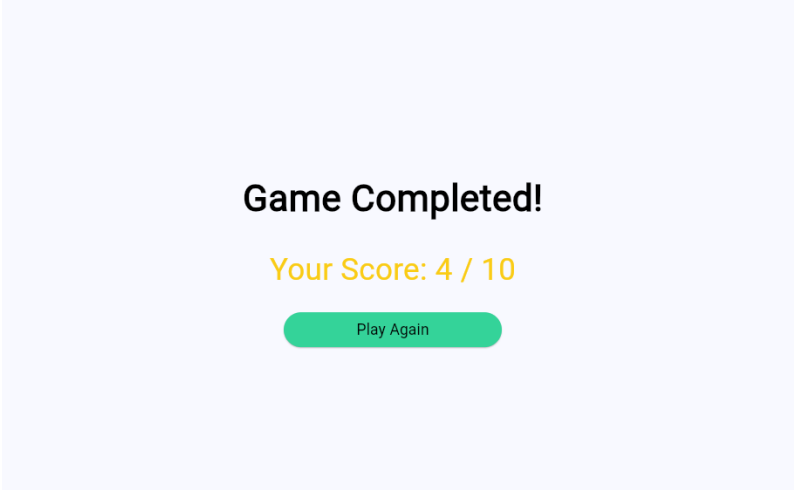
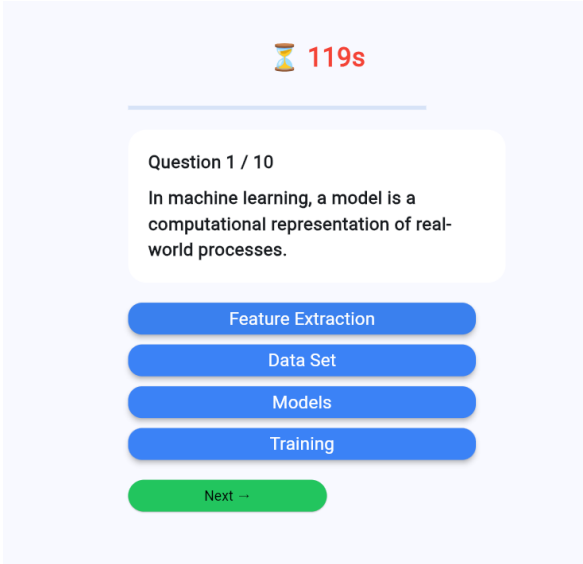
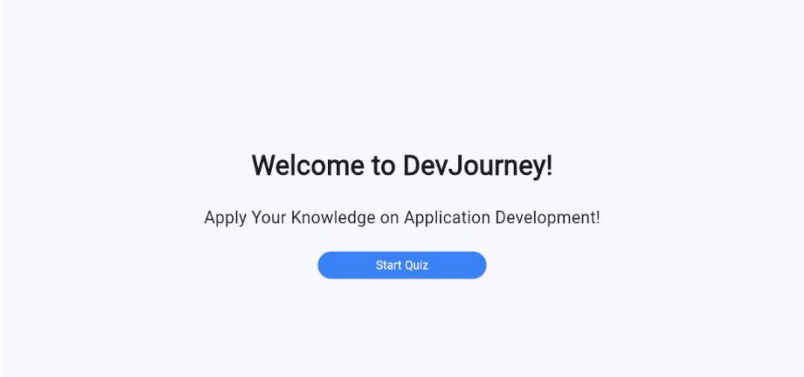
Name: Somera, Alord Jhinroy M.
Course and Section: BSCS 3-5

Date: 27/11/2025
Score:

GRADING RUBRICS:

CRITERIA	PERFORMANCE INDICATORS	POINTS
User Interface (UI) & Controls	Professional, responsive, and intuitive design using appropriate Flet controls.	25
Code Readability	Code is concise, easy to read, and well-structured with appropriate use of functions.	25
Completeness	Meets all specifications and functions correctly with expected input.	25
Logic	The app provides accurate outputs.	25
Total		100

SCREENSHOT of OUTPUT:




```

# Persistent top timer bar
timer_container = ft.Container(
    content=timer_text,
    alignment=ft.alignment.center,
    padding=10,
    visible=False # HIDDEN on welcome page
)

# Main content frame
content_container = ft.Column(expand=True, spacing=20)

# root container
root = ft.Column(
    [
        timer_container,
        content_container
    ],
    horizontal_alignment="center",
    alignment="start",
    spacing=20
)

page.add(root)

progress = ft.ProgressBar(width=300, value=0)

# Welcome Page
def show_welcome():
    content_container.controls = [
        ft.Column(
            ft.Column(
                [
                    ft.Text("Welcome to DevJourney!", size=32, weight="bold"),
                    ft.Text("Apply Your Knowledge on Application Development!", size=20),
                    ft.ElevatedButton(
                        "Start Quiz",
                        width=200,
                        bgcolor="#3b82f6",
                        color="white",
                        on_click=start_quiz
                    )
                ],
                alignment="center",
                horizontal_alignment="center",
                spacing=25
            )
        ]
    )
    content_container.update()
    page.update()

# Function that start the quiz
def start_quiz(e):
    timer_container.visible = True # Show timer when quiz begins
    page.run_task(update_timer) # Start timer
    show_question() # Show first question

# showing questions
def show_question():
    nonlocal selected

```

```

selected = None
q = QUESTIONS[index]

question_card = ft.Container(
    padding=20,
    width=380,
    bgcolor="white",
    border_radius=18,
    shadow=ft.BoxShadow(blur_radius=18, spread_radius=2, color="#00000030"),
    content=ft.Column(
        [
            ft.Text(f"Question {index + 1} / {len(QUESTIONS)}",
                    size=18, weight="bold", color="darkred"),
            ft.Text(q["Question"], size=18, weight="bold")
        ],
        spacing=10
    )
)

option_buttons = []

# Highlight the selected answer

def select_answer(e):
    nonlocal selected
    selected = e.control.content.value

    for btn in option_buttons:

```

```

        if btn.content.value == selected:
            btn.bgcolor = "#d81d1d" # dark blue highlight
            btn.color = "white"
            btn.elevation = 12
        else:
            btn.bgcolor = "#3b82f6" # default blue
            btn.color = "white"
            btn.elevation = 4

    page.update()

```

```

# Build option buttons
for opt in q["Options"]:
    btn = ft.ElevatedButton(
        content=ft.Text(opt, size=18),
        width=350,
        bgcolor="#3b82f6",
        color="white",
        on_click=select_answer,
        style=ft.ButtonStyle(
            shape=ft.RoundedRectangleBorder(radius=14),
            elevation=4
        )
    )
    option_buttons.append(btn)

```

```

next_btn = ft.ElevatedButton(
    "Next →",
    width=200,
    on_click=next_question,
    style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=18))
)

```

```

progress.value = index / len(QUESTIONS)

content_container.controls = [
    progress,
    question_card,
    ft.Column(option_buttons, spacing=10),
    next_btn
]

```

```

content_container.update()
page.update()

```

function that keeps the questions moving randomly

```

def next_question(e):
    nonlocal index, score, selected

    if selected == QUESTIONS[index]["Answer"]:
        score += 1

    index += 1

    if index < len(QUESTIONS):
        show_question()
    else:
        result_screen()

```

this is the result frame

```

def result_screen():
    timer_container.visible = False # Hide timer on result screen

    content_container.controls = [
        ft.Column(
            [
                ft.Text("Game Completed!", size=34, weight="bold", color="Black"),
                ft.Text(f"Your Score: {score} / {len(QUESTIONS)}",
                    size=28, color="#facc15"),
                ft.ElevatedButton(
                    "Play Again",
                    bgcolor="#34d399",
                    color="black",
                    width=200,
                    on_click=restart_game
                )
            ],
            alignment="center",
            horizontal_alignment="center",
            spacing=20
        )
    ]
    content_container.update()
    page.update()

```

restart game

```

def restart_game(e):
    nonlocal index, score, timer_value

```

```
        width=200,
        on_click=restart_game
    )
],
alignment="center",
horizontal_alignment="center",
spacing=20
)
]
content_container.update()
page.update()

# restart game
def restart_game(e):
    nonlocal index, score, timer_value
    index = 0
    score = 0
    timer_value = 120
    random.shuffle(QUESTIONS)

    show_welcome() # Return to welcome screen

show_welcome()

ft.app(target=main)
```